

PDS TECH

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 3 | AULA 03 -

INTRODUÇÃO AO GRAPHQL

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3

HANDS ON 4

SAIBA MAIS 5

O QUE VOCÊ VIU NESTA AULA? 10

REFERÊNCIAS 11

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Nesta aula, vamos entender os conceitos fundamentais do GraphQL, uma tecnologia moderna criada pela equipe do Facebook para trabalhar com dados. Vamos começar entendendo o que é o GraphQL, sua proposta de resolver as limitações comuns enfrentadas no desenvolvimento de APIs tradicionais e como ele se diferencia do modelo REST.

Você verá como o GraphQL permite consultas mais flexíveis, possibilitando que clientes solicitem exatamente os dados de que precisam, sem retornos desnecessários ou múltiplas requisições. Vamos discutir como isso impacta a eficiência e o desempenho das aplicações, especialmente em cenários onde a complexidade dos dados ou a variedade de dispositivos clientes são fatores importantes.

Além disso, analisaremos as diferenças estruturais entre o GraphQL e o REST. Abordaremos tópicos como a substituição de endpoints fixos por um único ponto de entrada, o uso de schemas para definir tipos e relações de dados e o papel das queries e mutations no lugar dos métodos HTTP tradicionais. Essas diferenças serão apresentadas não apenas como uma comparação técnica, mas também com uma visão prática, destacando as vantagens e desafios associados ao uso do GraphQL.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

Nesta aula, vamos aprofundar nosso entendimento sobre como o GraphQL funciona, explorando sua estrutura e lógica por meio de exemplos e análises detalhadas. Nosso foco será desvendar os componentes essenciais do GraphQL, desde sua organização até os mecanismos que tornam suas operações flexíveis e eficientes.

Começaremos examinando os schemas. Veremos como eles definem os tipos de dados, campos e relacionamentos, criando uma linguagem comum que conecta o cliente ao servidor. A partir daí, entenderemos como as queries permitem solicitar exatamente os dados necessários, demonstrando o diferencial do GraphQL em comparação com modelos mais tradicionais, como REST.

Avançando, analisaremos como o GraphQL trata as mutations, que possibilitam a alteração de dados no backend, como criar, atualizar ou excluir informações. Vamos discutir como essas operações são estruturadas e como elas diferem das requisições tradicionais que utilizam métodos HTTP, oferecendo maior controle e consistência.

Além disso, exploraremos o papel das resolvers, responsáveis por processar as solicitações feitas pelos clientes e entregar os resultados correspondentes. Essa arquitetura flexível do GraphQL será discutida para que você compreenda como ela se adapta a diferentes necessidades, seja para trabalhar com bancos de dados, APIs externas ou outras fontes de dados.

Também abordaremos como o GraphQL lida com relacionamentos entre dados, mostrando como consultas complexas podem ser realizadas em uma única requisição, eliminando a necessidade de múltiplos endpoints. Esse modelo reduz a sobrecarga e torna a interação entre cliente e servidor mais eficiente.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

Nesta aula vamos entender melhor sobre o tema de querys no GraphQL. No GraphQL, as queries são o principal mecanismo utilizado pelos clientes para solicitar dados ao servidor. Diferente dos métodos tradicionais em APIs REST, onde os endpoints retornam dados predefinidos, as queries no GraphQL oferecem uma abordagem mais flexível e precisa, permitindo que o cliente especifique exatamente quais informações deseja receber.

Uma query é estruturada de acordo com o schema definido pelo servidor, que atua como um contrato descrevendo os tipos de dados disponíveis, seus campos e relações. Por exemplo, se o schema define um tipo User com os campos id, name e email, o cliente pode solicitar apenas o nome e o email, sem a necessidade de receber dados adicionais desnecessários.

Como já vimos, esta é a estrutura de uma query:

```
query {
  user(id: 1) {
    name
    email
  }
}
```

Código-fonte 1 – Exemplo de query Graphql
Fonte: elaborado pelo autor (2024)

Essa query solicita ao servidor os campos name e email de um usuário específico identificado pelo id igual a 1. O servidor processa essa requisição e retorna apenas os dados solicitados, geralmente em um formato JSON:

```
{
  "data": {
    "user": {
      "name": "João Silva",
      "email": "joao.silva@email.com"
    }
  }
}
```

Código-fonte 2 – Exemplo de retorno para uma query Graphql
Fonte: elaborado pelo autor (2024)

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

As queries no GraphQL suportam a aninhamento (nested queries), permitindo que dados relacionados sejam recuperados em uma única requisição. Por exemplo, é possível buscar um usuário e os dados de seus pedidos associados:

```
query {
  user(id: 1) {
    name
    orders {
      id
      total
    }
  }
}
```

Código-fonte 3 – Exemplo de uma nested query Graphql
Fonte: elaborado pelo autor (2024)

Outro recurso importante das queries é o uso de argumentos e variáveis. Argumentos, como o id no exemplo acima, ajudam a refinar as solicitações, enquanto variáveis permitem maior reutilização e flexibilidade, especialmente em aplicações dinâmicas.

Além disso, o GraphQL oferece mecanismos para lidar com erros e respostas parciais. Mesmo que parte da query falhe, o servidor retorna os dados disponíveis com informações detalhadas sobre o erro, permitindo ao cliente gerenciar essas situações de forma eficiente. As queries têm o papel de empoderar o cliente para obter exatamente o que precisa, reduzindo o overhead de dados e a complexidade das interações.

Com as querys, temos as mutations. No GraphQL, as mutations são utilizadas para realizar alterações nos dados do servidor. Elas desempenham o papel que, em APIs REST, seria realizado pelos métodos HTTP como POST, PUT, PATCH e DELETE. Diferente das queries, que apenas leem dados, as mutations permitem criar, atualizar ou excluir informações, retornando, em geral, o resultado da operação.

Uma mutation segue a estrutura definida no schema do servidor, que especifica quais operações de escrita estão disponíveis, os argumentos necessários e os dados que podem ser retornados após a execução. Por exemplo, se o schema inclui uma mutation createUser, ela pode ser usada para adicionar um novo usuário ao sistema:

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
mutation {
  createUser(input: { name: "João Silva", email: "joao.silva@email.com" }) {
    id
    name
    email
  }
}
```

Código-fonte 4 – Exemplo de uma mutation Graphql
Fonte: elaborado pelo autor (2024)

No exemplo acima, a mutation createUser recebe um argumento input contendo os dados do novo usuário. Após a execução, o servidor retorna os campos especificados (id, name e email), confirmando a criação e permitindo que o cliente obtenha informações adicionais sem a necessidade de uma nova requisição.

Além da criação de dados, as mutations também são usadas para atualizações e exclusões. Por exemplo, uma mutation para atualizar o email de um usuário poderia ser estruturada assim:

```
mutation {
  updateUser(id: 1, input: { email: "novo.email@email.com" }) {
    id
    email
  }
}
```

Código-fonte 5 – Exemplo de uma mutation Graphql para atualizar um usuário
Fonte: elaborado pelo autor (2024)

Já uma mutation para excluir um usuário, da seguinte forma:

```
mutation {
  deleteUser(id: 1) {
    success
    message
  }
}
```

Código-fonte 5 – Exemplo de uma mutation Graphql para excluir um usuário
Fonte: elaborado pelo autor (2024)

Assim como nas queries, as mutations podem retornar respostas personalizadas, que ajudam a confirmar o sucesso da operação ou fornecer informações adicionais relevantes. Isso é uma grande vantagem em relação aos

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

métodos tradicionais de APIs REST, onde muitas vezes é necessário realizar múltiplas requisições para obter dados relacionados à operação executada.

Outro ponto importante é que, mesmo em operações de escrita, o GraphQL mantém sua estrutura declarativa. O cliente especifica claramente quais dados deseja modificar e quais informações espera como retorno, mantendo o controle sobre a interação.

Além disso, as mutations suportam operações complexas que envolvem múltiplas alterações simultâneas. Por exemplo, é possível criar uma ordem de compra e associar produtos a ela em uma única operação, simplificando a lógica de negócio no cliente e no servidor.

Assim como nas queries, o GraphQL oferece suporte robusto a tratamento de erros em mutations. Isso garante que falhas durante a execução sejam devidamente reportadas, permitindo ao cliente identificar e corrigir problemas rapidamente.

O GraphQL, com suas querys e mutations, se destaca em aplicações que precisam atender a diferentes tipos de dispositivos, como aplicativos móveis, web ou até dispositivos IoT. Ele permite que cada cliente solicite exatamente os dados que precisa, evitando o envio de informações desnecessárias e otimizando a experiência do usuário.

Imagine, por exemplo, uma aplicação onde um aplicativo móvel precisa apenas de dados essenciais para telas pequenas, enquanto a versão web requer informações mais detalhadas e completas. Com o GraphQL, as aplicações podem se conectar ao mesmo ponto de entrada e realizar consultas adaptadas a suas necessidades específicas, sem a necessidade de criar múltiplos endpoints no servidor.

Além disso, o GraphQL é extremamente útil em sistemas onde os dados possuem relações complexas. Em vez de realizar várias requisições para obter informações relacionadas, como seria comum em APIs REST, o GraphQL permite que todas as informações necessárias sejam recuperadas em uma única consulta estruturada. Isso é especialmente relevante em domínios como e-commerce, onde os clientes podem solicitar dados sobre produtos, categorias e avaliações em um único pedido, ou em plataformas sociais, onde é possível obter detalhes de usuários, postagens e interações de forma integrada.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Ele também se aplica a situações em que as mudanças nos requisitos de dados são frequentes. Em projetos com evoluções constantes, adicionar novos campos ou ajustar os existentes no GraphQL não afeta os clientes que utilizam os campos anteriores. Essa abordagem orientada a schemas torna o sistema mais resiliente a mudanças, reduzindo a necessidade de criar versões distintas da API.

Outro caso relevante para o uso do GraphQL é em equipes que desejam melhorar a experiência de pessoas desenvolvedoras. O GraphQL fornece uma interface de exploração interativa, como o GraphiQL ou Apollo Studio, que facilita a descoberta dos tipos de dados e suas relações, simplificando o desenvolvimento e reduzindo o tempo necessário para integrar novos consumidores.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos os conceitos fundamentais do GraphQL e compreendemos como ele se diferencia das abordagens tradicionais, como o REST. Analisamos o papel do GraphQL como uma solução moderna para o consumo eficiente de dados, destacando sua capacidade de permitir que os clientes solicitem exatamente as informações necessárias, sem retornos excessivos ou múltiplas requisições.

Entendemos como o schema atua como a base da API, definindo os tipos de dados disponíveis e suas relações, servindo como contrato entre cliente e servidor. Vimos como as queries permitem acessar os dados com precisão e flexibilidade, eliminando a necessidade de endpoints rígidos e facilitando consultas que integram informações relacionadas em uma única requisição.

Também discutimos as mutations, que possibilitam operações de escrita no back-end, como criação, atualização ou exclusão de dados, garantindo que o cliente possa executar essas ações de maneira estruturada e obter respostas detalhadas sobre os resultados. Essa abordagem foi analisada em comparação com os métodos tradicionais, destacando os benefícios em termos de clareza e eficiência.

Além disso, abordamos os principais casos de uso do GraphQL, entendendo onde ele se aplica melhor, como em sistemas que atendem a múltiplos dispositivos, ambientes com dados complexos e cenários onde a evolução contínua da API é uma necessidade.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

GRAPHQL**. Introduction to GraphQL**. 2024. Disponível em: <https://graphql.org/learn/>. Acesso em: 09 dez. 2024.

GRAPHQL**. Queries and Mutations**. 2024. Disponível em: <https://graphql.org/learn/queries/>. Acesso em: 09 dez. 2024.

GRAPHQL. **Schemas and Types**. 2024. Disponível em: <https://graphql.org/learn/schema/>. Acesso em: 09 dez. 2024.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

GraphQL. API Flexível. Consultas Personalizadas. Schema Definition Language (SDL). Resolvers. Eficiência em Dados. Overfetching. Underfetching. Microsserviços. GraphQL Server. GraphiQL. Integração Cliente-Servidor. Tipos Fortemente Tipados. Fragmentos. Diretivas. Subscriptions. Mutations. Query Language. N+1 Problem. DataLoader. Documentação Dinâmica. Ferramentas Dev. Cloud-Native. APIs Escaláveis. Apollo. Relay. Prisma. Integração Fullstack. Depuração. Versionamento Suave. Segurança em APIs.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com